Graph Data Structure

Definitions, Terminology, Types, Representations and Traversal

Data Structures and Algorithms
Undergraduate and GATE Preparation

Contents

1 Introduction to Graphs **3**
1.1 Example Graph . . . 3
1.2 Applications of Graphs . . . 3

2 Basic Graph Terminology **4**
2.1 Vertex . . . 4
2.2 Edge . . . 4
2.3 Adjacent Vertices . . . 4
2.4 Incident Edge . . . 4
2.5 Degree of a Vertex . . . 4
2.6 Handshaking Lemma . . . 5
2.7 In-Degree and Out-Degree . . . 5
2.8 Walk, Trail and Path . . . 6
2.9 Cycle . . . 6
2.10 Connected Graph . . . 6
2.11 Disconnected Graph . . . 6
2.12 Isolated Vertex . . . 6
2.13 Pendant Vertex . . . 6
2.14 Loop and Parallel Edges . . . 6
2.15 Subgraph . . . 6
2.16 Bridge . . . 7
2.17 Articulation Point . . . 7

3 Types of Graphs **7**
3.1 Undirected Graph . . . 7
3.2 Directed Graph . . . 7
3.3 Weighted Graph . . . 7
3.4 Simple Graph . . . 8
3.5 Multigraph . . . 8
3.6 Pseudograph . . . 8
3.7 Complete Graph . . . 8

3.8 Regular Graph . . . 9
3.9 Null Graph . . . 9
3.10 Connected Graph . . . 9
3.11 Disconnected Graph . . . 9
3.12 Cyclic Graph . . . 9
3.13 Acyclic Graph . . . 9
3.14 Bipartite Graph . . . 9
3.15 Directed Acyclic Graph . . . 10
3.16 Planar Graph . . . 10

4 Graph Representation **10**
4.1 Adjacency Matrix . . . 10
4.2 Adjacency List . . . 11
4.3 Representation Comparison . . . 11

5 Graph Traversal **11**
5.1 Example Graph . . . 11

6 Breadth-First Search **12**
6.1 Definition . . . 12
6.2 BFS Algorithm . . . 12
6.3 Step-by-Step BFS . . . 12
6.4 BFS Levels . . . 13
6.5 Applications of BFS . . . 13

7 Depth-First Search **13**
7.1 Definition . . . 13
7.2 DFS Algorithm . . . 14
7.3 Step-by-Step DFS . . . 14
7.4 Applications of DFS . . . 14

8 Comparison of BFS and DFS **15**

9 Important Graph Formulas **16**

10 Solved Numerical Problems **16**

11 Practice Questions **18**

12 Answers to Numerical Questions **18**

13 Conclusion **19**

1. Introduction to Graphs

A graph is a non-linear data structure consisting of a finite set of vertices and a collection of edges representing relationships among them.

Mathematically, a graph is represented as

$$\boxed{G = (V, E)}$$

where

- V is the set of vertices or nodes.
- E is the set of edges connecting vertices.

For an undirected graph, an edge is an unordered pair of vertices.

For a directed graph, an edge is an ordered pair of vertices.

1.1 Example Graph

Consider

$$V = \{A, B, C, D\}$$

and

$$E = \{AB, AC, BC, BD, CD\}.$$

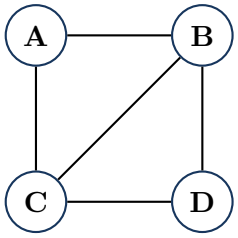


Figure 1: An undirected graph with four vertices and five edges.

The number of vertices is

$$|V| = 4.$$

The number of edges is

$$|E| = 5.$$

1.2 Applications of Graphs

Graphs are widely used in:

1. Computer networks and routing

2. Social network analysis
3. Road networks and navigation
4. Web page linking and crawling
5. Task scheduling and dependencies
6. Recommendation systems
7. Electrical and communication networks
8. Artificial intelligence and search

2. Basic Graph Terminology

2.1 Vertex

A vertex is a fundamental element of a graph representing an entity or an object.
For example, in a social network, a vertex represents a user.

2.2 Edge

An edge represents a relationship between two vertices.
In an undirected graph,

$$e = \{u, v\}.$$

In a directed graph,

$$e = (u, v).$$

2.3 Adjacent Vertices

Two vertices are adjacent if they are connected by an edge.
For example, if edge AB exists, then A and B are adjacent.

2.4 Incident Edge

An edge is incident on each of its endpoint vertices.
If $e = \{A, B\}$, then e is incident on both A and B.

2.5 Degree of a Vertex

The degree of a vertex in an undirected graph is the number of incident edge ends.
A loop contributes two to the degree.
It is represented by

$$\deg(v).$$

Consider the example graph in Section 1.

$$\deg(A) = 2$$

$$\deg(B) = 3$$

$$\deg(C) = 3$$

$$\deg(D) = 2.$$

Thus,

$$\sum_{v \in V} \deg(v) = 2 + 3 + 3 + 2 = 10.$$

2.6 Handshaking Lemma

For every finite undirected graph,

$$\boxed{\sum_{v \in V} \deg(v) = 2|E|}$$

Since $|E| = 5$,

$$2|E| = 2(5) = 10.$$

Therefore, the handshaking lemma is verified.

Important result:

The number of vertices having odd degree is always even.

2.7 In-Degree and Out-Degree

For a directed graph:

In-degree $\deg^-(v)$ is the number of incoming edges.

Out-degree $\deg^+(v)$ is the number of outgoing edges.

$$\boxed{\sum_{v \in V} \deg^-(v) = |E|}$$

$$\boxed{\sum_{v \in V} \deg^+(v) = |E|}$$

2.8 Walk, Trail and Path

Walk: A sequence of vertices and edges where repetition is allowed.

Trail: A walk in which no edge is repeated.

Path: A walk in which no vertex is repeated.

For example,

$$A \to B \to C$$

is a path when edges AB and BC exist.

2.9 Cycle

A cycle is a closed path having no repeated vertices other than the first and last vertex.

For example,

$$A \to B \to C \to A.$$

2.10 Connected Graph

An undirected graph is connected if a path exists between every pair of vertices.

2.11 Disconnected Graph

An undirected graph is disconnected if at least two vertices have no path between them.

2.12 Isolated Vertex

A vertex with degree zero is called an isolated vertex.

$$\boxed{\deg(v) = 0}$$

2.13 Pendant Vertex

A vertex having degree one is called a pendant vertex.

$$\boxed{\deg(v) = 1}$$

2.14 Loop and Parallel Edges

A loop is an edge connecting a vertex to itself.

Parallel edges are multiple edges between the same pair of vertices.

2.15 Subgraph

A graph $H = (V_H, E_H)$ is a subgraph of $G = (V, E)$ if

$$V_H \subseteq V$$

and

$$E_H \subseteq E.$$

2.16 Bridge

A bridge is an edge whose removal increases the number of connected components of an undirected graph.

2.17 Articulation Point

An articulation point is a vertex whose removal, along with its incident edges, increases the number of connected components.

3. Types of Graphs

3.1 Undirected Graph

An undirected graph contains edges without orientation.

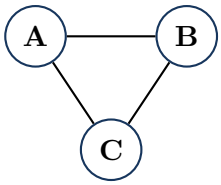


Figure 2: Undirected graph.

For an edge $\{u, v\}$, traversal is possible in both directions.

3.2 Directed Graph

A directed graph contains edges having a specific direction.

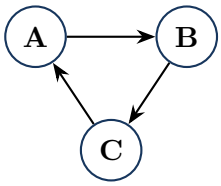


Figure 3: Directed graph.

3.3 Weighted Graph

A weighted graph assigns a numerical value to each edge.

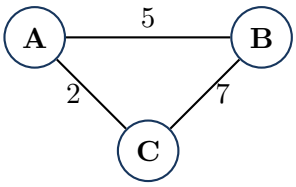


Figure 4: Weighted graph.

Weights may represent distance, time, cost or capacity.

3.4 Simple Graph

A simple graph contains no loops and no parallel edges.

3.5 Multigraph

A multigraph permits parallel edges between vertices.

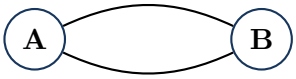


Figure 5: Multigraph with parallel edges.

3.6 Pseudograph

A pseudograph permits loops and may contain parallel edges.

3.7 Complete Graph

A complete simple undirected graph contains an edge between every pair of distinct vertices. It is represented by K_n.

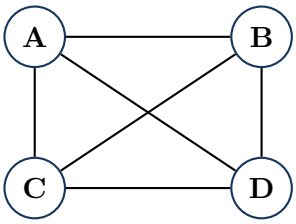


Figure 6: Complete graph K_4.

The number of edges in K_n is

$$\boxed{|E| = \frac{n(n-1)}{2}}.$$

For K_4,

$$|E| = \frac{4(3)}{2} = 6.$$

3.8 Regular Graph

A graph is k-regular if every vertex has degree k.

$$\deg(v) = k, \qquad \forall v \in V.$$

3.9 Null Graph

A null graph contains vertices but no edges.

$$E = \varnothing.$$

3.10 Connected Graph

Every pair of vertices is connected by a path.

3.11 Disconnected Graph

A disconnected graph contains more than one connected component.

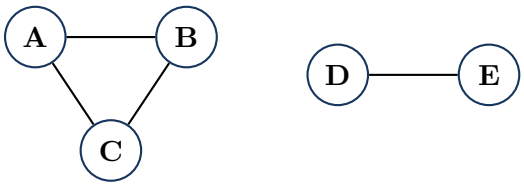


Figure 7: Disconnected graph with two connected components.

3.12 Cyclic Graph

A cyclic graph contains at least one cycle.

3.13 Acyclic Graph

An acyclic graph contains no cycles.

A connected acyclic undirected graph is called a tree.

3.14 Bipartite Graph

A graph is bipartite if its vertices can be partitioned into two disjoint sets U and W such that every edge has one endpoint in each set.

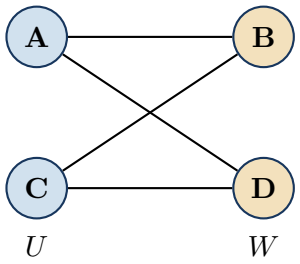


Figure 8: Bipartite graph.

An undirected graph is bipartite if and only if it contains no odd-length cycle.

3.15 Directed Acyclic Graph

A directed acyclic graph (DAG) is a directed graph containing no directed cycle.

DAGs are commonly used in task scheduling and dependency management.

3.16 Planar Graph

A planar graph is a graph that can be drawn in the plane without edge crossings except at common endpoints.

4. Graph Representation

Graphs are commonly represented using an adjacency matrix or an adjacency list.

Consider

$$V = \{A, B, C, D\}$$

and

$$E = \{AB, AC, BC, CD\}.$$

4.1 Adjacency Matrix

An adjacency matrix is a two-dimensional array.

For a simple unweighted graph,

$$M[i][j] = \begin{cases} 1, & \text{if an edge exists,} \\ 0, & \text{otherwise.} \end{cases}$$

	A	B	C	D
A	0	1	1	0
B	1	0	1	0
C	1	1	0	1
D	0	0	1	0

For an undirected graph, the adjacency matrix is symmetric.

$$M[i][j] = M[j][i].$$

4.2 Adjacency List

In an adjacency list, each vertex stores its neighboring vertices.

```
A -> B, C
B -> A, C
C -> A, B, D
D -> C
```

4.3 Representation Comparison

Property	Matrix	List
Space	$O(V^2)$	$O(V+E)$
Edge lookup	$O(1)$	$O(\deg(v))$
Neighbor traversal	$O(V)$	$O(\deg(v))$
BFS complexity	$O(V^2)$	$O(V+E)$
DFS complexity	$O(V^2)$	$O(V+E)$
Best suited for	Dense	Sparse

The adjacency-list edge lookup bound shown above applies to an undirected graph without an auxiliary lookup index.

5. Graph Traversal

Graph traversal is the systematic process of visiting vertices and exploring the edges of a graph.

The two fundamental methods are:

1. Breadth-First Search (BFS)
2. Depth-First Search (DFS)

5.1 Example Graph

Consider the graph

$$V = \{A, B, C, D, E, F\}$$

with

$$E = \{AB, AC, BD, BE, CF\}.$$

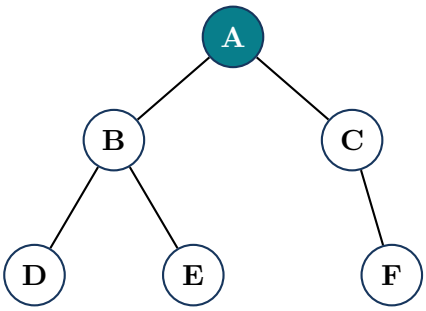


Figure 9: Graph used for BFS and DFS traversal.

Throughout this example, adjacent vertices are visited alphabetically.

6. Breadth-First Search

6.1 Definition

Breadth-First Search is a graph traversal algorithm that explores vertices level by level using a queue (FIFO).

6.2 BFS Algorithm

```python
from collections import deque

def bfs(graph, start):
    visited = {start}
    queue = deque([start])

    while queue:
        vertex = queue.popleft()
        print(vertex, end=" ")

        for neighbor in graph[vertex]:
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append(neighbor)
```

6.3 Step-by-Step BFS

Start BFS at vertex A.

Step	Processed	Queue Afterwards
1	A	B, C
2	B	C, D, E
3	C	D, E, F
4	D	E, F
5	E	F
6	F	Empty

Thus, the BFS traversal order is

$$\boxed{A \to B \to C \to D \to E \to F}.$$

6.4 BFS Levels

$$L_0 = \{A\}$$

$$L_1 = \{B, C\}$$

$$L_2 = \{D, E, F\}.$$

Vertex	BFS Distance
A	0
B	1
C	1
D	2
E	2
F	2

6.5 Applications of BFS

1. Shortest path in an unweighted graph
2. Connected components
3. Undirected cycle detection
4. Bipartite graph checking
5. BFS spanning trees
6. Level calculation
7. Network broadcasting

$$\boxed{T(V, E) = O(V + E)}$$

for adjacency lists.

7. Depth-First Search

7.1 Definition

Depth-First Search explores a path as deeply as possible before backtracking to explore alternative paths.
DFS uses recursion or an explicit stack.

7.2 DFS Algorithm

```python
def dfs(graph, vertex, visited=None):
    if visited is None:
        visited = set()

    visited.add(vertex)
    print(vertex, end=" ")

    for neighbor in graph[vertex]:
        if neighbor not in visited:
            dfs(graph, neighbor, visited)
```

7.3 Step-by-Step DFS

Start from A and visit adjacent vertices alphabetically.

1. Visit A.
2. Move from A to B.
3. Move from B to D.
4. Backtrack to B.
5. Visit E.
6. Backtrack to A.
7. Visit C.
8. Finally, visit F.

Therefore,

$$\boxed{A \to B \to D \to E \to C \to F}.$$

7.4 Applications of DFS

1. Connected components
2. Strongly connected components
3. Cycle detection
4. Topological sorting
5. Articulation points
6. Bridges
7. Biconnected components
8. DFS spanning trees
9. Discovery and finishing times
10. Backtracking

$$\boxed{T(V, E) = O(V + E)}$$

for adjacency lists.

8. Comparison of BFS and DFS

Parameter	BFS	DFS
Data structure	Queue	Stack/Recursion
Traversal strategy	Level-wise	Depth-wise
Unweighted shortest path	Guaranteed	Not guaranteed
Connected components	Yes	Yes
Cycle detection	Yes	Yes
Bipartite checking	Yes	Yes
Spanning tree	BFS tree	DFS tree
Topological sorting	Kahn's algorithm	DFS finishing order
Articulation points	Not standard BFS	Low-link DFS
Time complexity	$O(V + E)$	$O(V + E)$
Auxiliary space	$O(V)$	$O(V)$

For a disconnected graph, one BFS or DFS starting from a single vertex visits only the vertices reachable from that vertex.
To traverse the entire graph, repeat traversal from each remaining unvisited vertex.

9. Important Graph Formulas

1. **Handshaking Lemma**

$$\boxed{\sum_{v \in V} \deg(v) = 2E}$$

2. **Maximum Edges in a Simple Undirected Graph**

$$\boxed{E_{\max} = \frac{V(V-1)}{2}}$$

3. **Maximum Edges in a Simple Directed Graph Without Loops**

$$\boxed{E_{\max} = V(V-1)}$$

4. **Number of Edges in a Tree**

$$\boxed{E = V - 1}$$

5. **Number of Forest Edges**

$$\boxed{E_{\text{forest}} = V - C}$$

6. **Non-Tree Edges in an Undirected Graph**

$$\boxed{E_{\text{non-tree}} = E - V + C}$$

7. **Total In-Degree and Out-Degree**

$$\boxed{\sum_{v} \deg^{-}(v) = \sum_{v} \deg^{+}(v) = E}$$

8. **BFS/DFS Complexity**

$$\boxed{O(V + E)}$$

10. Solved Numerical Problems

Problem 1: Maximum Edges

A simple undirected graph contains 8 vertices. Find the maximum possible number of edges.

Solution:

$$E_{\max} = \frac{V(V-1)}{2}.$$

$$E_{\max} = \frac{8(7)}{2}.$$

$$\boxed{E_{\max} = 28}.$$

Problem 2: Handshaking Lemma

An undirected graph has degree sequence

$$(3, 3, 2, 2, 2).$$

Find the number of edges.

Solution:

$$\sum \deg(v) = 3 + 3 + 2 + 2 + 2 = 12.$$

Using the handshaking lemma,

$$2E = 12.$$

$$\boxed{E = 6}.$$

Problem 3: Spanning Forest

An undirected graph contains 15 vertices and 3 connected components.

Find the number of edges in a spanning forest.

Solution:

$$V - C = 15 - 3.$$

$$\boxed{12}.$$

Problem 4: Non-Tree Edges

An undirected graph contains 10 vertices, 15 edges and 2 connected components.

Find the number of non-tree edges.

Solution:

$$E_{\text{non}-\text{tree}} = E - V + C.$$

$$= 15 - 10 + 2.$$

$$\boxed{7}.$$

Problem 5: Directed Graph Degree

A directed graph has 12 edges.

Find the total in-degree and total out-degree.

Solution:

$$\boxed{\sum_v \deg^-(v) = 12}$$

and

$$\boxed{\sum_v \deg^+(v) = 12}.$$

The combined total is 24.

11. Practice Questions

Q1. Define a graph and explain its basic components.

Q2. Differentiate between directed and undirected graphs with suitable examples.

Q3. Explain degree, in-degree and out-degree of a vertex.

Q4. State and prove the handshaking lemma.

Q5. Differentiate between a walk, trail, path and cycle.

Q6. Define complete, regular, bipartite and connected graphs.

Q7. Explain adjacency matrix and adjacency list representations.

Q8. Perform BFS and DFS on a graph of six vertices starting from vertex A.

Q9. A simple undirected graph contains 12 vertices. Find the maximum possible number of edges.

Q10. An undirected graph contains 20 vertices and 4 connected components. Find the number of edges in a spanning forest.

12. Answers to Numerical Questions

Question 9:

$$E_{\max} = \frac{12(11)}{2}.$$

$$\boxed{66}.$$

Question 10:

$$E_{\text{forest}} = V - C.$$

$$= 20 - 4.$$

$$\boxed{16}.$$

13. Conclusion

Graphs are fundamental non-linear data structures used to represent relationships between objects.

Important graph concepts include vertices, edges, degree, paths, cycles, connectivity and different graph classifications.

Graphs can be represented using adjacency matrices or adjacency lists.

Breadth-First Search and Depth-First Search are the two fundamental graph traversal algorithms.

BFS explores vertices level by level, whereas DFS explores deeply before backtracking.

When adjacency lists are used, both algorithms run in

$$\boxed{O(V + E)}$$

time.

These fundamental concepts form the basis for advanced graph algorithms, including shortest paths, minimum spanning trees, strongly connected components, articulation points, bridges and topological sorting.